SWCON221 Microservice Programming

Lecture 9

Docker Compose

2026. 09. 01

Sungwon Lee

Department of Software Convergence

경희대학교
KYUNG HEE UNIVERSITY

Contents

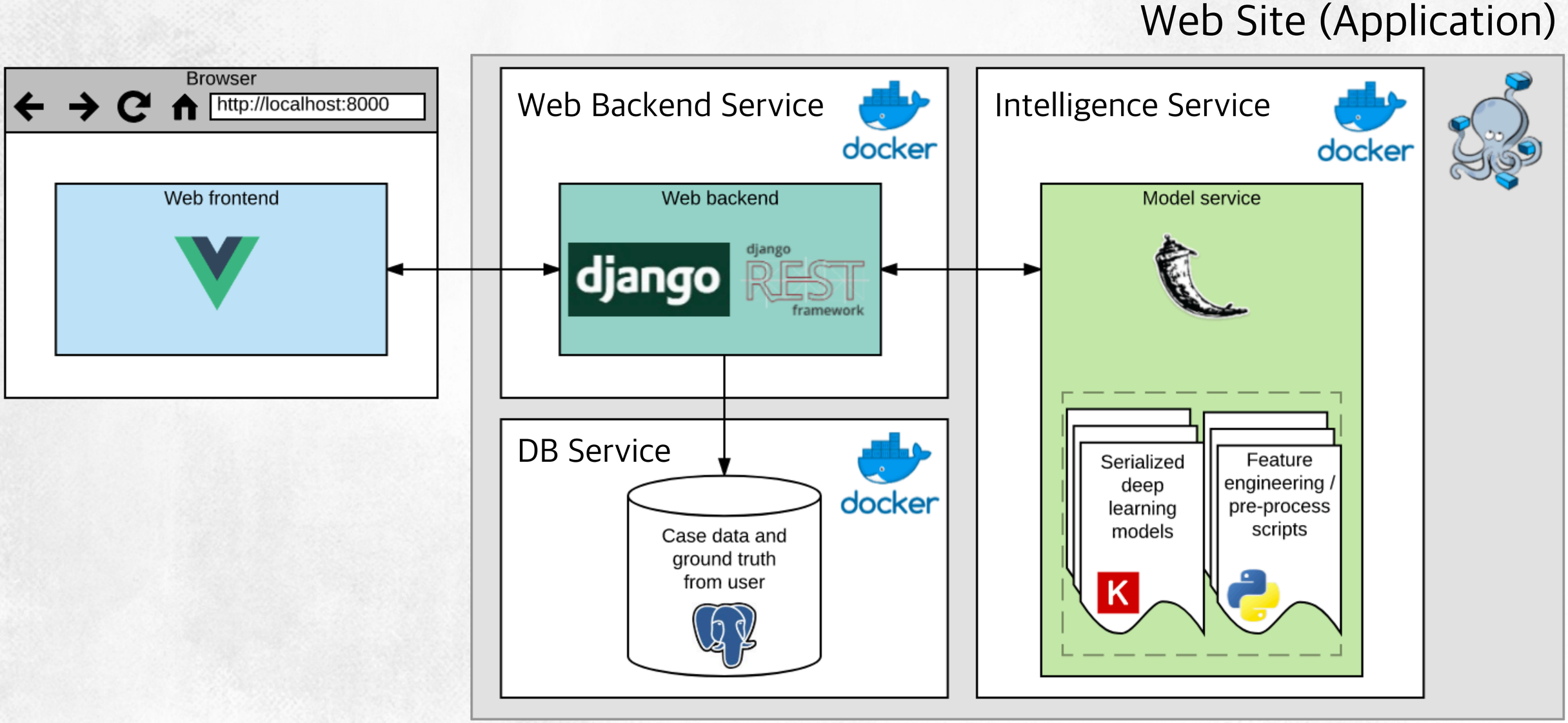
- Services
- Docker Compose Basic
- Docker Compose Installation
- Docker Compose Files
- Docker Compose Test Drive #1~#2
- YAML Overall
- Docker Compose Test Drive #3~#5
- FYI

Docker Services

- Different pieces of the app are called “services”,
  - in a distributed application
- For example, if you imagine a video sharing site:
  - a service for storing application data in a database
  - a service for video transcoding in the background after a user uploads something
  - a service for the front-end,
  - and so on.

Docker Services

- Example: General Web Site Application & Services



Reference: http://drivendata.co/blog/docker-for-concept-to-clinic-challenge/

Docker Services

- Services are really just "containers in production"
  - a service only runs one image,
  - but it codifies the way that image runs
    - What ports it should use
    - How many replicas of the container should run so the service has the capacity it needs, and so on
- Scaling a service changes the number of container instances running that piece of software
  - Assigning more computing resources to the service in the process

What is Docker Compose?

- A tool for defining and running multi-container docker applications
- Use a YAML file to configure application services
  - docker-compose.yml

Docker Compose Features

- Start all services

  docker compose up

  (*previously,* docker-compose *command*)

- Stop all services

  docker compose down

- Scale up selected services when required

  docker compose up --scale

Installation of Docker Compose

docker docs | Search the docs | Home | Guides | Manuals | Reference | Samples

Manuals / Docker Compose / Install Compose / Install Docker Compose

- Docker Desktop
- Docker Engine
- Docker Compose
  - Overview of Docker Compose
  - Install Compose
    - Install Docker Compose
    - Install Compose plugin through Desktop
    - Install Docker Compose Plugin
    - Uninstall Docker Compose
  - Getting started
  - Environment variables in Compose
  - Environment file
  - Using service profiles
  - GPU support in Compose
  - Extend services in Compose
  - Networking in Compose
  - Using Compose in production
  - Control startup order

Edit this page

Request docs changes

On this page:

- Compose prerequisites
- Compose installation scenarios
- Where to go next

Install Docker Compose

On this page you can find a summary of the available options for installing Docker Compose.

Compose prerequisites

- Docker Compose requires Docker Engine.
- Docker Compose plugin requires Docker CLI.

Compose installation scenarios

You can run Compose on macOS, Windows, and 64-bit Linux. Check what installation scenario fits your needs.

Are you looking to:

- **Get latest Docker Compose and its prerequisites**: Install Docker Desktop for your platform. This is the fastest route and you get Docker Engine and Docker CLI with the Compose plugin. Docker Desktop is available for Mac, Windows and Linux.
- **Install Compose plugin:**
  - **(Mac, Win, Linux) Docker Desktop**: If you have Desktop installed then you already have the Compose plugin installed.
  - **Linux systems**: To install the Docker CLI's Compose plugins use one of these methods of installation:
    - Using the convenience scripts offered per Linux distro from the Engine install section.
    - Setting up Docker's repository and using it to install the compose plugin package.
    - Other scenarios, check the Linux install.
  - **Windows Server**: If you want to run the Docker daemon and client directly on Microsoft Windows Server, follow the Windows Server install instructions.

Reference: https://docs.docker.com/compose/install/

Installation of Docker Compose

dockerdocs Get started Guides Manuals Reference Gordon Search

AI AND AGENTS
- Overview
- Docker Sandboxes
- MCP Catalog and Toolkit Beta
- Model Runner
- Gordon
- AI and Docker Compose
- Docker Agent

APPLICATION DEVELOPMENT
- Docker Desktop
- Docker Engine
- Docker Offload
- Docker Build
- Docker Build Cloud
- Docker Compose
  - Introduction to Compose
  - Install
    - Plugin
    - Standalone (Legacy)
    - Uninstall
  - Quickstart

Home / Manuals / Docker Compose / Install

Overview of installing Docker Compose

Ask Gordon Copy Markdown View Markdown

This page summa
depending on yo

Installatio

Docker Deskt

The easiest and
Docker Desktop.

Docker Desktop i
Docker CLI which

Docker Desktop i

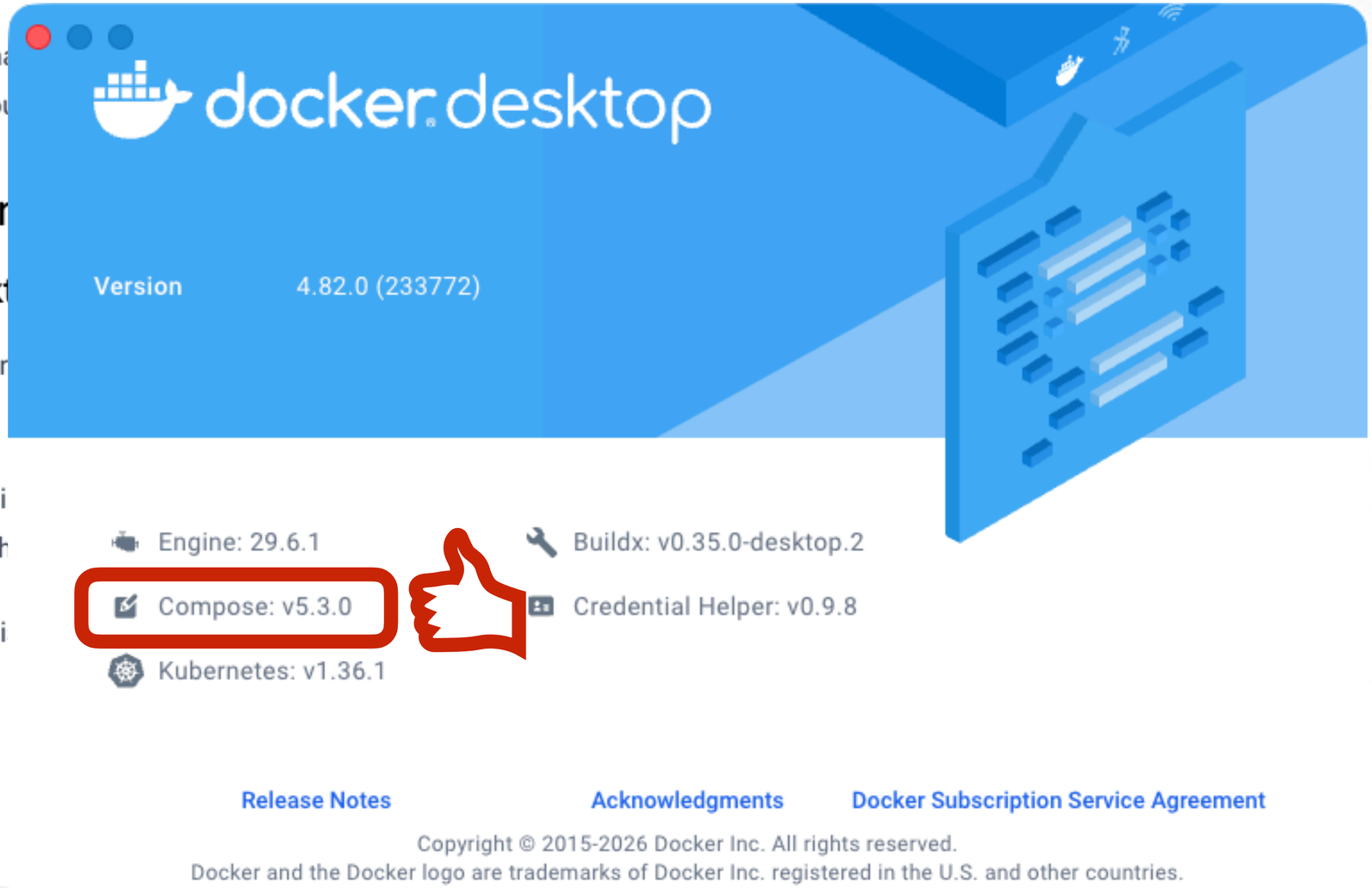
- Linux
- Mac
- Windows

Edit this page

Request changes

Table of contents

Installation scenarios



Release Notes Acknowledgments Docker Subscription Service Agreement

Copyright © 2015-2026 Docker Inc. All rights reserved.
Docker and the Docker logo are trademarks of Docker Inc. registered in the U.S. and other countries.

Reference: https://docs.docker.com/compose/install/

Installation of Docker Compose

- Check of docker-compose has been installed

  docker compose

  docker compose --help

  docker compose version

How to create docker compose files

- Make your working directory
- Move into your working directory
- Create docker-compose.yml file
- Fill the contents
  - WARN: TAB is not allowed !!!

```
version: '3.7'
services:
  web:
    image: nginx
  database:
    image: redis
```

```
version: '3.3'

services:
   db:
     image: mysql:5.7
     volumes:
       - db_data:/var/lib/mysql
     restart: always
     environment:
       MYSQL_ROOT_PASSWORD: somewordpress
       MYSQL_DATABASE: wordpress
       MYSQL_USER: wordpress
       MYSQL_PASSWORD: wordpress

   wordpress:
     depends_on:
       - db
     image: wordpress:latest
     ports:
       - "8000:80"
     restart: always
     environment:
       WORDPRESS_DB_HOST: db:3306
       WORDPRESS_DB_USER: wordpress
       WORDPRESS_DB_PASSWORD: wordpress
       WORDPRESS_DB_NAME: wordpress
volumes:
    db_data: {}
```


Reference: https://docs.docker.com/samples/wordpress/

How to create docker compose files

- Compose file versions
  - ~~Recommend : 3.9~~ >> Changed to INFORMATIVE !!!

Version top-level element

Top-level `version` property is defined by the specification for backward compatibility but is only informative.

A Compose implementation SHOULD NOT use this version to select an exact schema to validate the Compose file, but prefer the most recent schema at the time it has been designed.

Compose implementations SHOULD validate whether they can fully parse the Compose file. If some fields are unknown, typically because the Compose file was written with fields defined by a newer version of the specification, Compose implementations SHOULD warn the user. Compose implementations MAY offer options to ignore unknown fields (as defined by "loose" mode).

Reference: https://docs.docker.com/compose/compose-file/

HOMEWORK

How to create services

- docker compose up -d

docker-compose.yml

```yaml
version: '3.7'
services:
  web:
    image: nginx
  database:
    image: redis
```

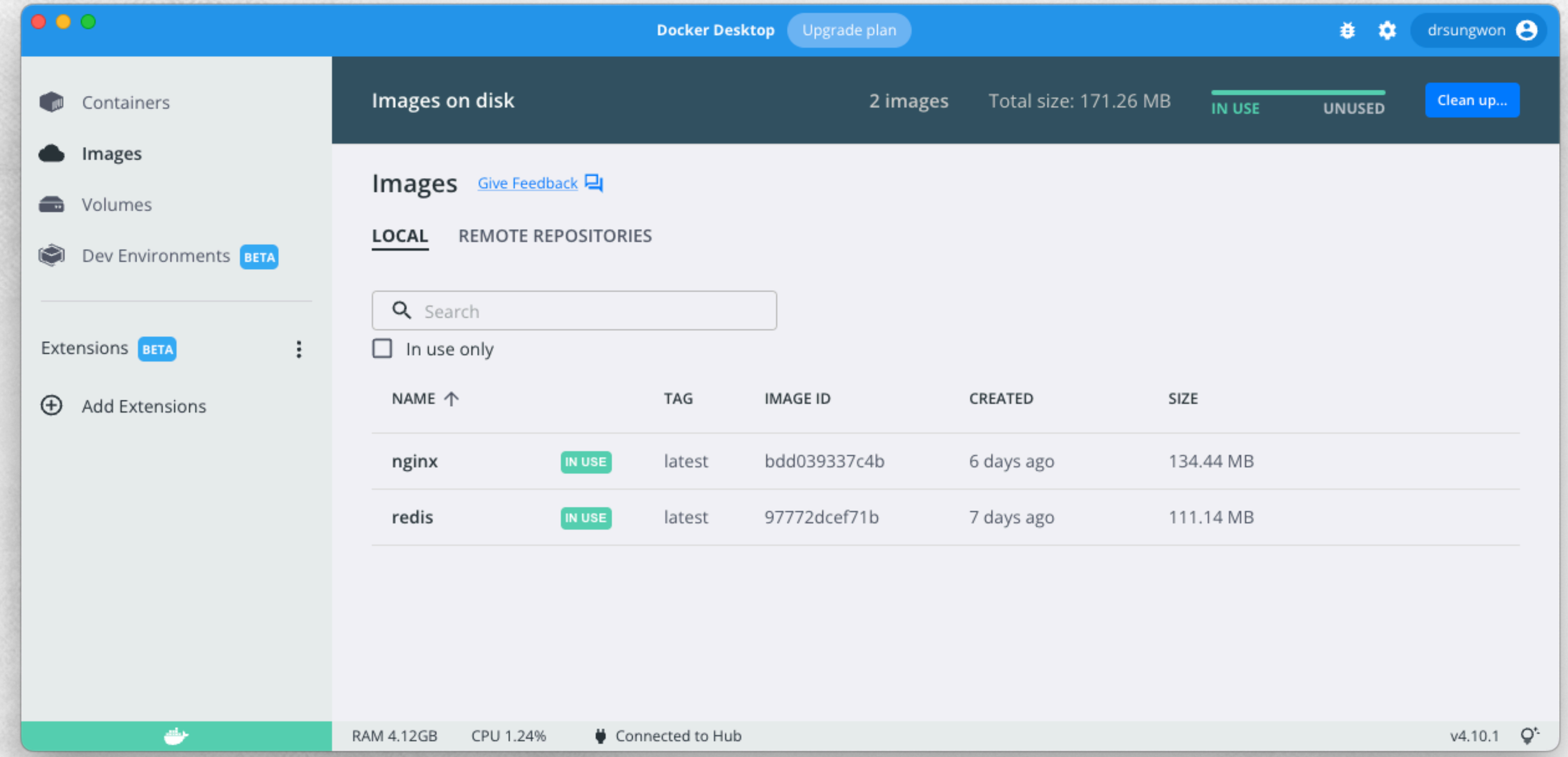
docker-compose execution

```
drsungwon~$ docker-compose up -d
[+] Running 13/13
 ⠿ database Pulled
   ⠿ 60197a4c18d4 Pull complete
   ⠿ 649d52ef4fd9 Pull complete
   ⠿ 15708d2d5b0f Pull complete
   ⠿ 7563f50a936f Pull complete
   ⠿ 9f69cdd1652c Pull complete
   ⠿ 33cf3eea7301 Pull complete
 ⠿ web Pulled
   ⠿ 452bd3c050dc Pull complete
   ⠿ 549e784aa15b Pull complete
   ⠿ d3af141b830d Pull complete
   ⠿ 9d3bf1abc70f Pull complete
   ⠿ 4914f84d0b6d Pull complete
[+] Running 3/3
 ⠿ Network lecture09-sources_default        Created
 ⠿ Container lecture09-sources-web-1        Started
 ⠿ Container lecture09-sources-database-1   Started
drsungwon~$
```

HOMEWORK

Check activated services (GUI)

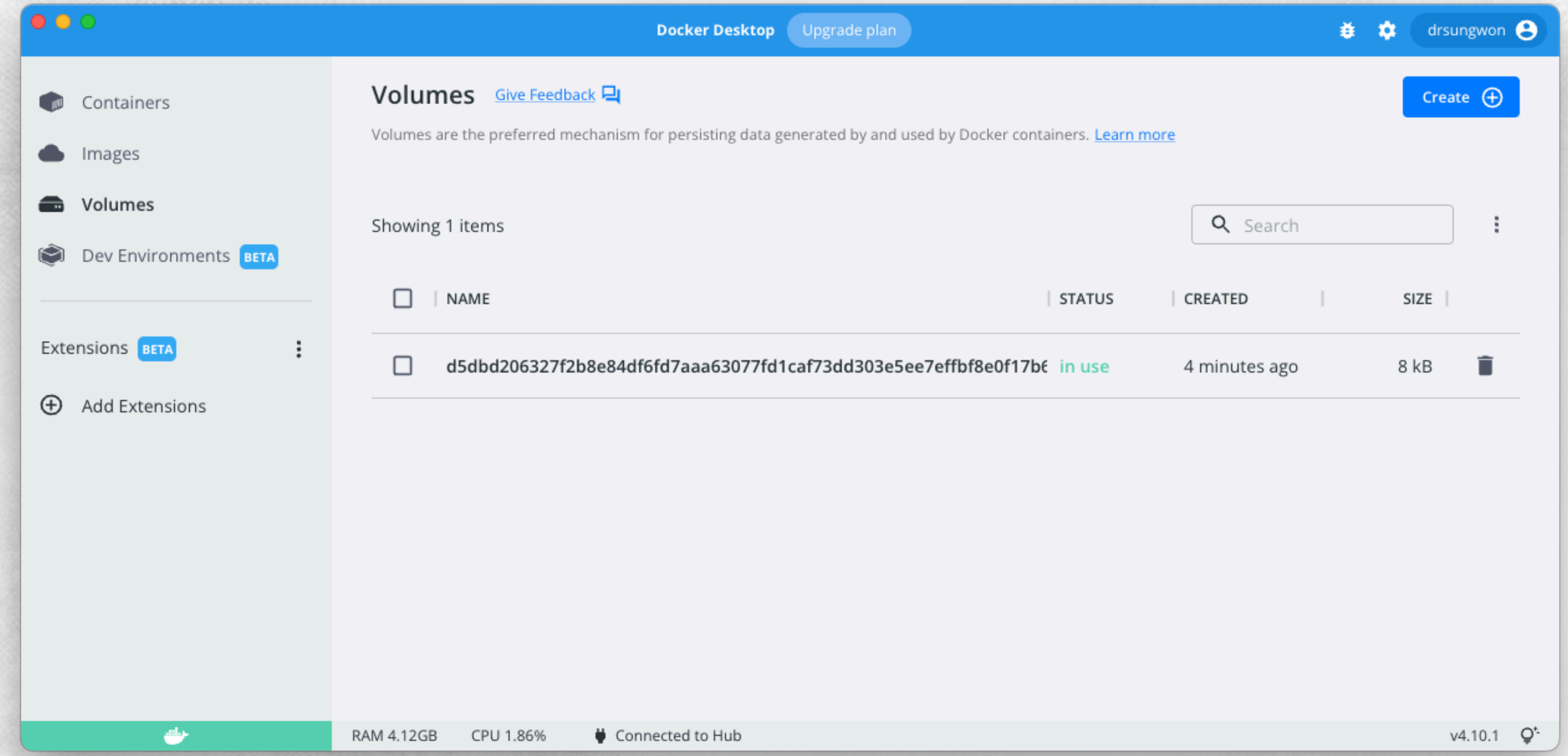
- Images



HOMEWORK

Check activated services (GUI)

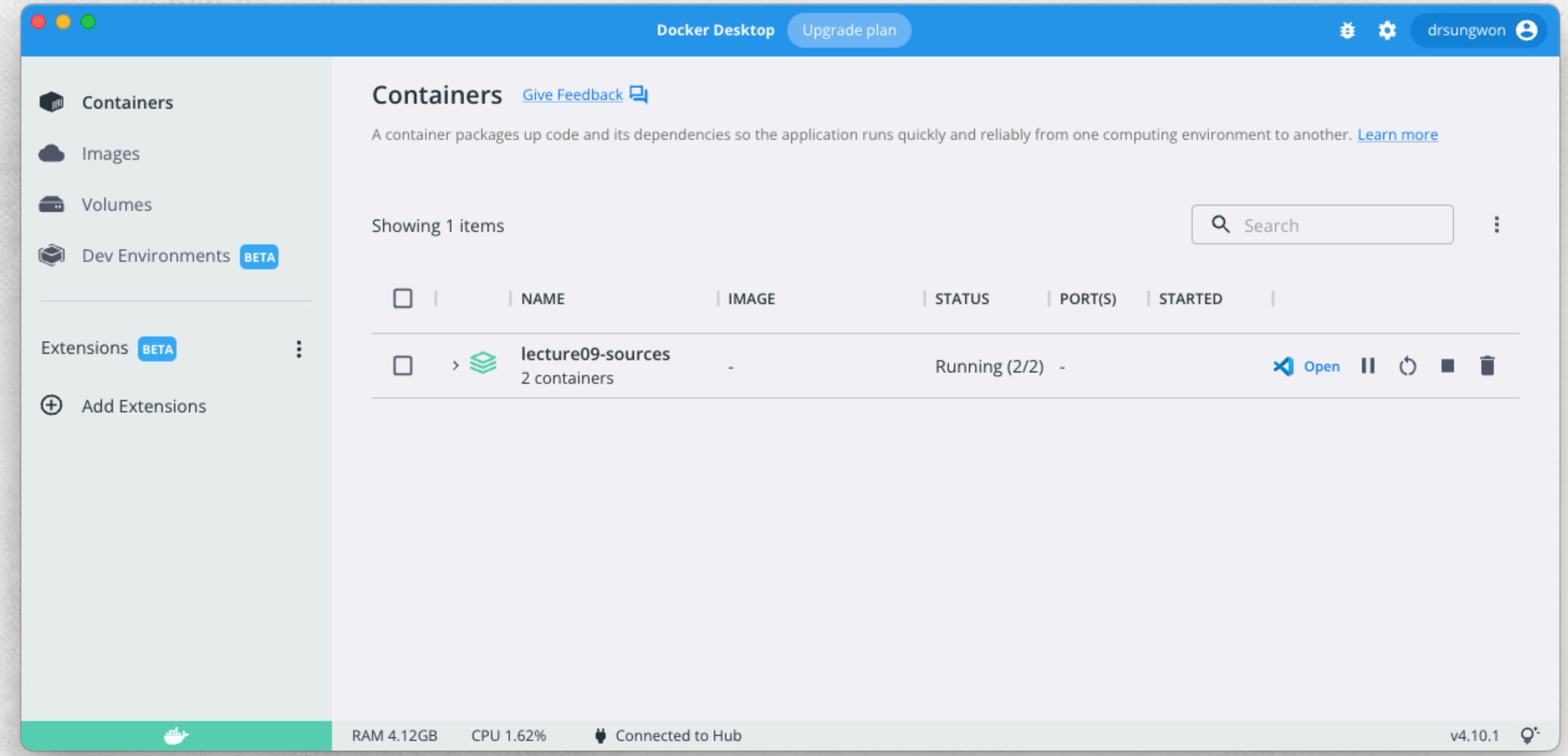
- Volumes



HOMEWORK

Check activated services (GUI)

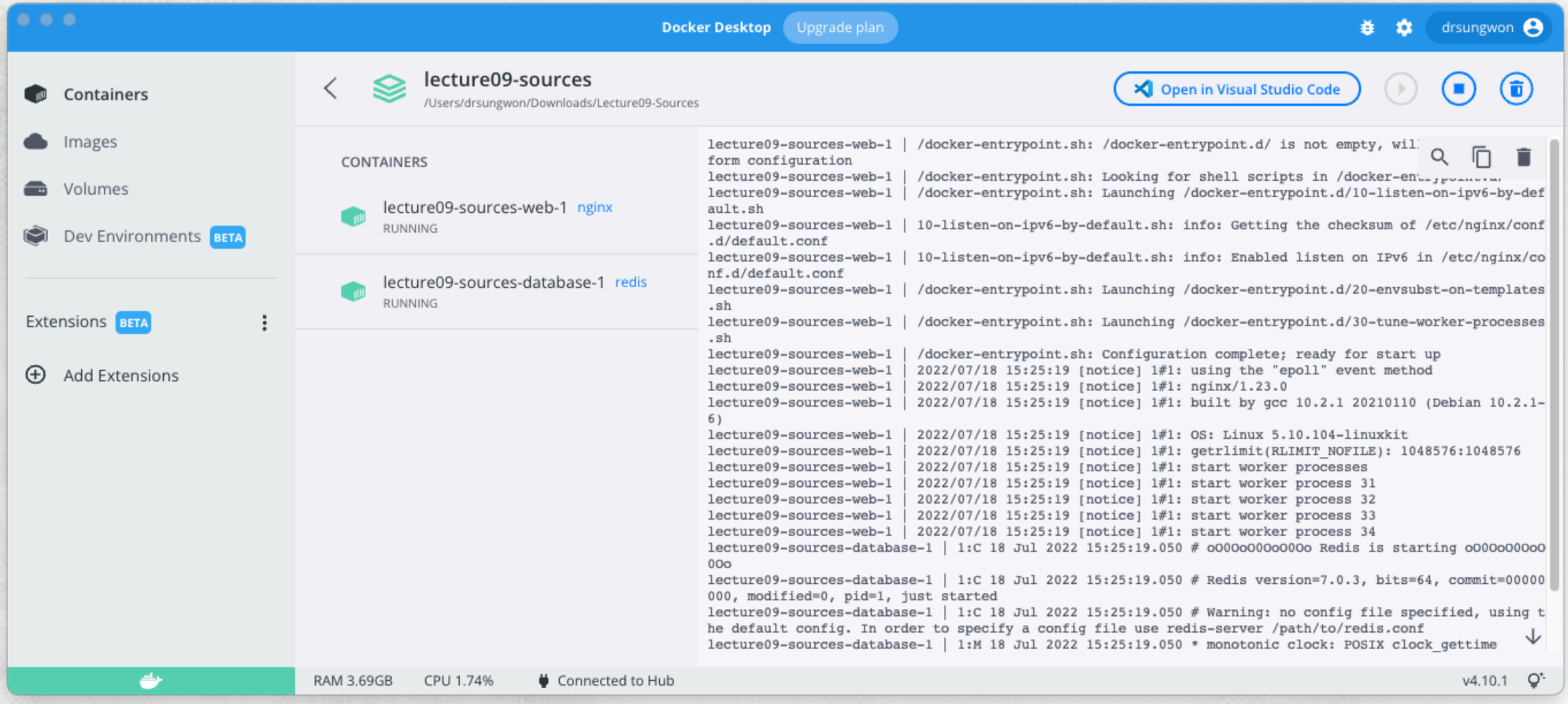
- Containers



HOMEWORK

Check activated services (GUI)

- Containers - Detail Information



Check activated services (CLI)

- docker compose config

```
[MacMini-ServerFarm:testdrive drsungwon$ docker-compose config
services:
  database:
    image: redis
  web:
    image: nginx
version: '3.7'

[MacMini-ServerFarm:testdrive drsungwon$ docker-compose config --help
Validate and view the Compose file.

Usage: config [options]

Options:
    --resolve-image-digests  Pin image tags to digests.
    -q, --quiet              Only validate the configuration, don't print
                             anything.
    --services               Print the service names, one per line.
    --volumes                Print the volume names, one per line.
    --hash="*"               Print the service config hash, one per line.
                             Set "service1,service2" for a list of specified services
                             or use the wildcard symbol to display all services
[MacMini-ServerFarm:testdrive drsungwon$ docker-compose config --services
web
database
[MacMini-ServerFarm:testdrive drsungwon$ docker-compose config --volumes

MacMini-ServerFarm:testdrive drsungwon$
```

Check activated services (CLI)

- docker container ls -a & docker image ls

```
MacMini-ServerFarm:testdrive drsungwon$ docker container ls -a
CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS              PORTS               NAMES
1dcf2c0e7e43        redis               "docker-entrypoint.s…"   8 minutes ago       Up 8 minutes        6379/tcp            testdrive_database_1
c49838e950db        nginx               "nginx -g 'daemon of…"   8 minutes ago       Up 8 minutes        80/tcp              testdrive_web_1
MacMini-ServerFarm:testdrive drsungwon$ docker image ls
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
redis               latest              01a52b3b5cd1        8 days ago          98.2MB
nginx               latest              f949e7d76d63        9 days ago          126MB
MacMini-ServerFarm:testdrive drsungwon$
```

- docker stats

```
CONTAINER ID        NAME                CPU %               MEM USAGE / LIMIT     MEM %               NET I/O             BLOCK I/O           PIDS
1dcf2c0e7e43        testdrive_database_1   0.23%               1.598MiB / 1.952GiB   0.08%               1.93kB / 0B         0B / 0B             4
c49838e950db        testdrive_web_1        0.00%               1.871MiB / 1.952GiB   0.09%               1.86kB / 0B         123kB / 0B          2
```

How to stop services (CLI)

- docker compose down

```
[MacMini-ServerFarm:testdrive drsungwon$ docker-compose down
Stopping testdrive_database_1 ... done
Stopping testdrive_web_1      ... done
Removing testdrive_database_1 ... done
Removing testdrive_web_1      ... done
Removing network testdrive_default
[MacMini-ServerFarm:testdrive drsungwon$
[MacMini-ServerFarm:testdrive drsungwon$
[MacMini-ServerFarm:testdrive drsungwon$ docker container ls -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS               NAMES
[MacMini-ServerFarm:testdrive drsungwon$ docker image ls
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
redis               latest              01a52b3b5cd1        8 days ago          98.2MB
nginx               latest              f949e7d76d63        9 days ago          126MB
MacMini-ServerFarm:testdrive drsungwon$
```

[주의: up 위치. 즉, yml 화일이 있는 곳에서 실행]

HOMEWORK

Modified using Port Mapping

- docker compose up -d

docker-compose.yml

```yaml
version: '3.7'
services:
  web:
    image: nginx
    ports:
      - "8000:80"
  database:
    image: redis
```

docker-compose execution

```
MacMini-ServerFarm:testdrive drsungwon$ docker-compose up -d
Creating network "testdrive_default" with the default driver
Creating testdrive_web_1      ... done
Creating testdrive_database_1 ... done
MacMini-ServerFarm:testdrive drsungwon$ curl http://localhost:8000
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
MacMini-ServerFarm:testdrive drsungwon$
```

HOMEWORK

Modified using Port Mapping

- docker compose up -d

docker-compose.yml

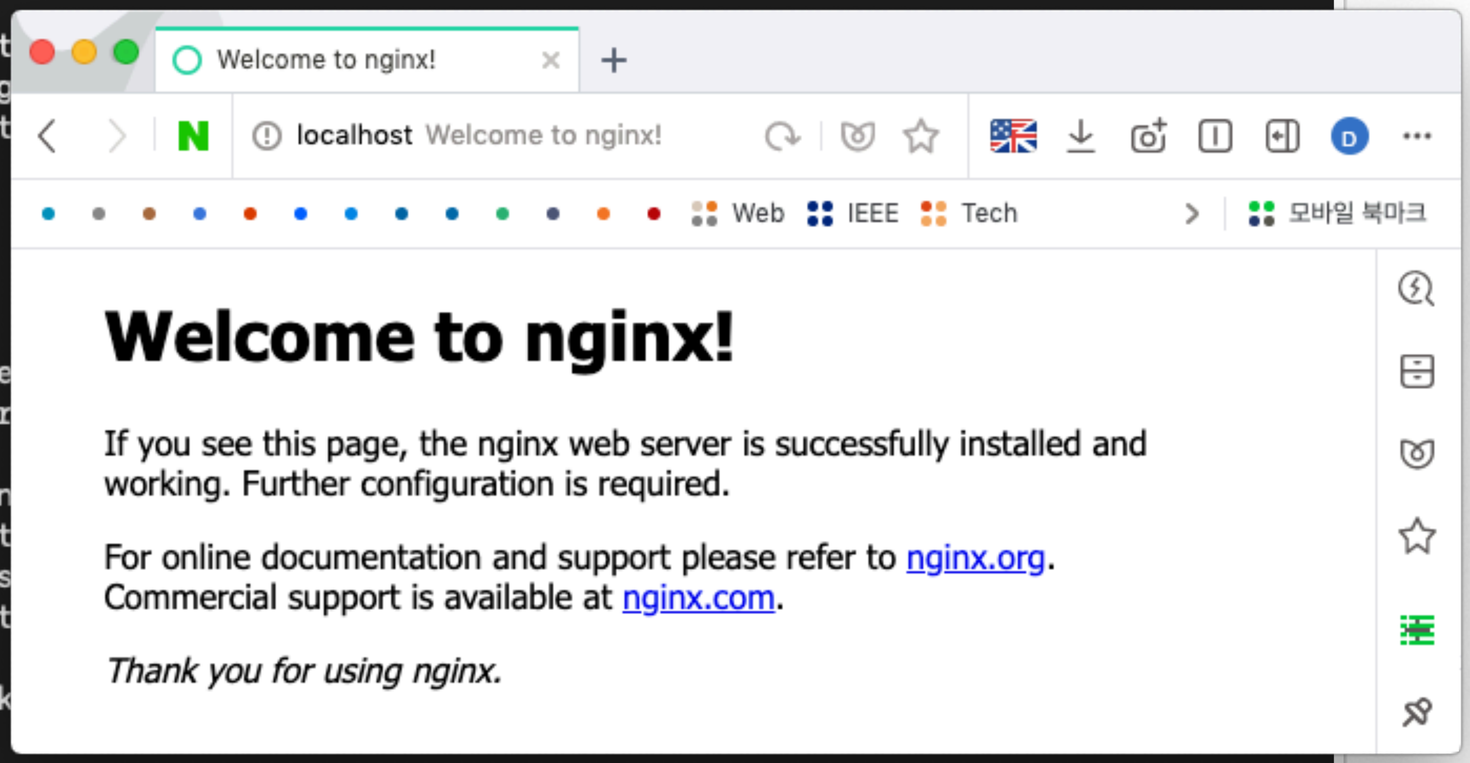
```yaml
version: '3.7'
services:
  web:
    image: nginx
    ports:
      - "8000:80"
  database:
    image: redis
```

docker-compose execution

```
MacMini-ServerFarm:testdrive drsungwon$ docker-compose up -d
Creating network "testdrive_default" with the default driver
Creating testdrive_web_1      ... done
Creating testdrive_database_1 ... done
MacMini-ServerFarm:testdrive drsungwon$ curl http://localhost:8000
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        widt
        marg
        font
    }
</style>
</head>
<body>
<h1>Welcome
<p>If you se
working. Fur

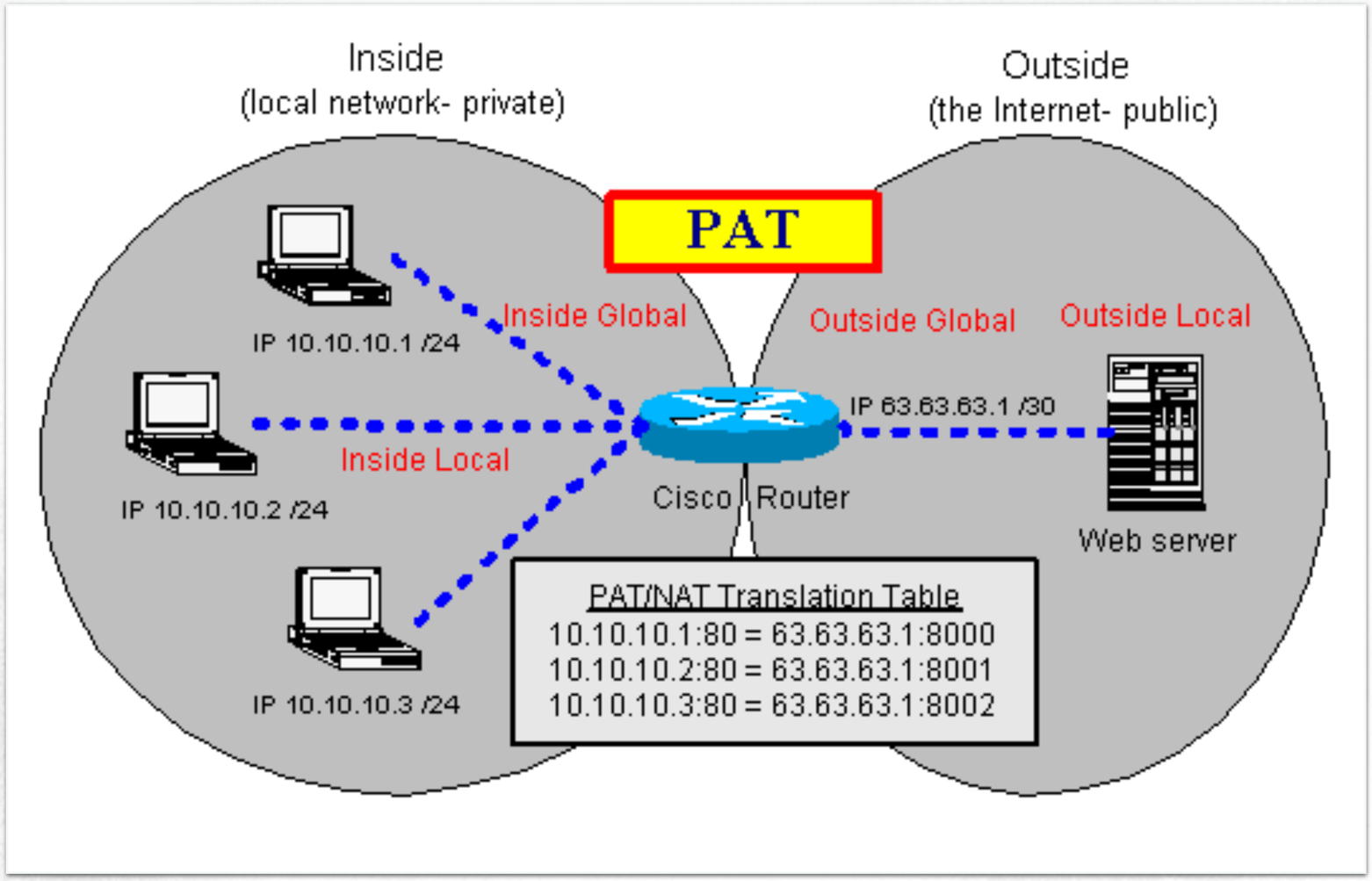
<p>For onlin
<a href="htt
Commercial s
<a href="htt

<p><em>Thank
</body>
</html>
MacMini-ServerFarm:testdrive drsungwon$
```



Create modified services using Port Mapping

- NAT/PAT
  - NAT: Network Address Translation
  - PAT: Port Address Translation



Reference: https://www.quora.com/What-is-Port-Address-Translation-PAT

YAML: YAML Ain't Markup Language

- A data serialization language designed to be human-friendly
- Work well with modern programming languages for common everyday tasks
- Indentation(= space)-based scoping
  - Similar to Python's
  - Facilitates easy inspection of the data's structure

YAML Use Cases

- Configuration files
- Log files
- Cross-language data sharing
- Debugging of complex data structures

yaml.org

- Standard & Resources

```
%YAML 1.2
---
YAML: YAML Ain't Markup Language

What It Is: YAML is a human friendly data serialization
  standard for all programming languages.

YAML Resources:
  YAML 1.2 (3rd Edition): http://yaml.org/spec/1.2/spec.html
  YAML 1.1 (2nd Edition): http://yaml.org/spec/1.1/
  YAML 1.0 (1st Edition): http://yaml.org/spec/1.0/
  YAML Issues Page:       https://github.com/yaml/yaml/issues
  YAML Mailing List:      yaml-core@lists.sourceforge.net
  YAML IRC Channel:       "#yaml on irc.freenode.net"
  YAML Cookbook (Ruby):   http://yaml4r.sourceforge.net/cookbook/ (local)
  YAML Reference Parser:  http://ben-kiki.org/ypaste/
  YAML Test Suite:        https://github.com/yaml/yaml-test-suite
```

yaml.org

- YAML libraries for most languages

```
Projects:
  C/C++ Libraries:
  - libyaml
  - libcyaml
  - Syck
  - yaml-cpp
  Crystal:
  - YAML
  Ruby:
  - psych
  - RbYaml
  Python:
  - PyYAML
  - ruamel.yaml
  - PySyck
  - JvYaml
  - SnakeYAML
  - YamlBeans
  - JYaml
  - Camel
  Perl Modules:
  - YAML
  - YAML::XS
  - YAML::Syck
  - YAML::Tiny
  - YAML::PP
  - PlYaml
  C#/.NET:
  - YamlDotNet
  - yaml-net
  - yatools.net
  Golang:
  - Go-yaml
  - Go-gypsy
  PHP:
  - The Yaml Component
  - php-yaml
  - syck
  - spyc
  OCaml:
  - ocaml-syck
  Javascript:
  - JS-YAML
  - JS-YAML Online          # Browserified JS-YAML demo, to play with YAML in your browser.
  Actionscript:
  - as3yaml                 # port of JvYAML (1.1)
  Haskell:
  - yaml                    # YAML parser/renderer for Haskell (based on libyaml)
  - HsYAML                  # YAML 1.2 implementation in pure Haskell
  - YamlReference           # Haskell 1.2 reference parser
  Dart:
  - yaml                    # YAML package for Dart
  Rust:
  - yaml-rust               # YAML 1.2 implementation in pure Rust
  - serde-yaml              # YAML de/serialization of structs
  Nim:
  - NimYAML                 # YAML 1.2 implementation in pure Nim
  Delphi:
  - Neslib.Yaml             # YAML 1.1 Delphi binding to libyaml
  Others:
  - yamlvim (src)           # YAML dumper/emitter in pure vimscript
```

Welcome to PyYAML

PyYAML is a full-featured YAML framework for the Python programming language.

PyYAML Resources

- PyYAML Documentation
- LibYAML is a YAML parser and emitter writte
- PyYAML Repository
- PyYAML Issues
- PyYAML Development Chat - #pyyaml irc.fr
- PyYAML Recent Release Downloads
- PyYAML Legacy Release Downloads

PyYAML Documentation

PyYAML is a YAML parser and emitter for Python.

Installation

Simple install:

```
pip install pyyaml
```

To install from source, download the source package *PyYAML-5.1.tar.gz* and unpack it. Go to the directory *PyYAML-5.1* and run:

```
$ python setup.py install
```

If you want to use LibYAML bindings, which are much faster than the pure Python version, you need to download and install LibYAML. Then you may build and install the bindings by executing

```
$ python setup.py --with-libyaml install
```

In order to use LibYAML based parser and emitter, use the classes `CParser` and `CEmitter`. For instance,

```
from yaml import load, dump
try:
    from yaml import CLoader as Loader, CDumper as Dumper
except ImportError:
    from yaml import Loader, Dumper

# ...

data = load(stream, Loader=Loader)

# ...

output = dump(data, Dumper=Dumper)
```

Note that there are some subtle (but not really significant) differences between pure Python and LibYAML based parsers and emitters.

HOMEWORK

Manager & Workers (Ubuntu) - 1/4

- docker-compose.yml

```yaml
version: '3.7'
services:
  manager:
    image: ubuntu:latest # https://hub.docker.com/_/ubuntu
    tty: true # https://docs.docker.com/engine/reference/run/
  worker1:
    image: nginx # https://hub.docker.com/_/nginx
    ports: # https://docs.docker.com/compose/compose-file/#ports
      - "8001:80"
  worker2:
    image: nginx # https://hub.docker.com/_/nginx
    ports: # https://docs.docker.com/compose/compose-file/#ports
      - "8002:80"
```

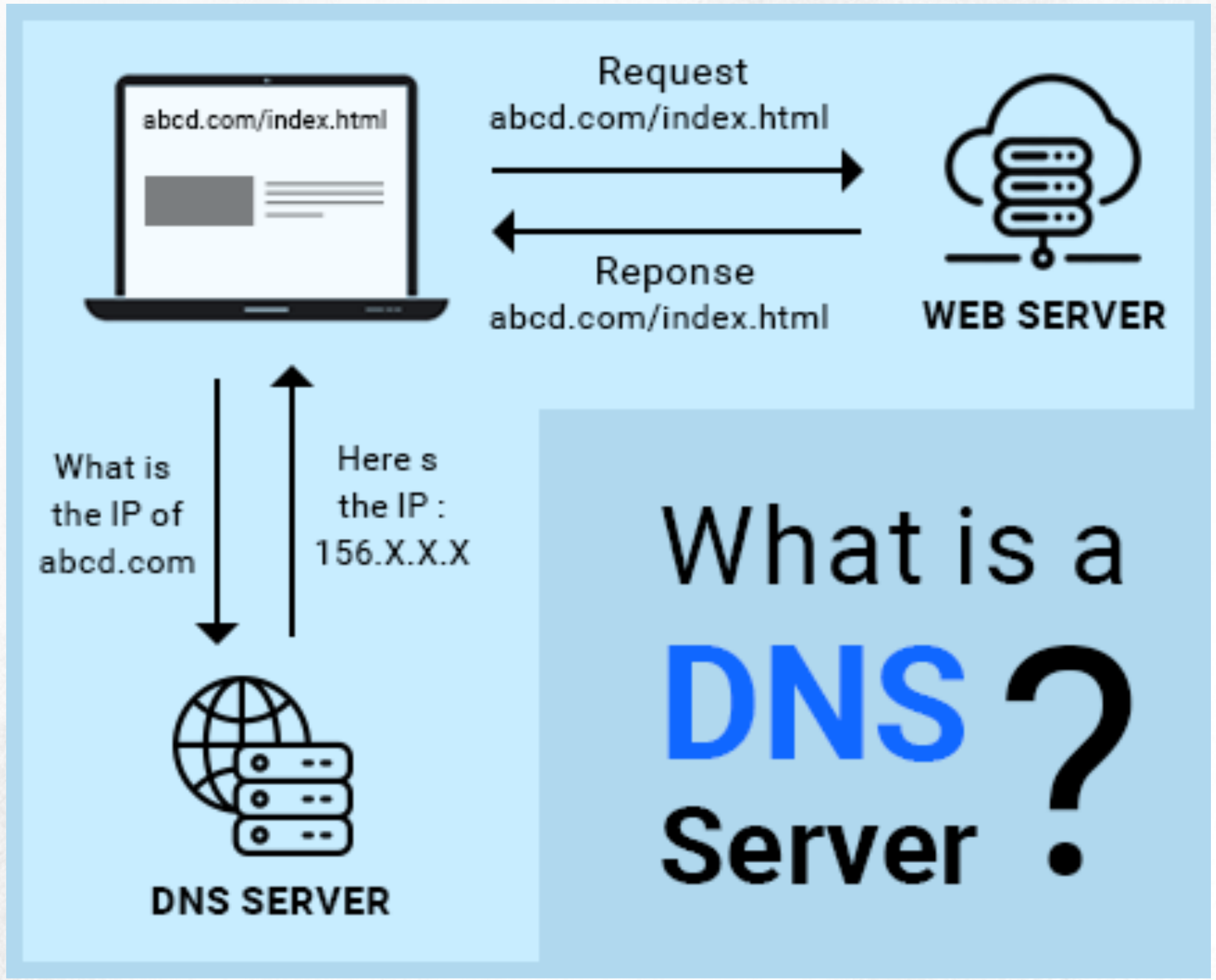
HOMEWORK

Manager & Workers (Ubuntu) - 2/4

- docker compose up -d
- docker container ls
- curl (or web browser) http://localhost:8001
- curl (or web browser) http://localhost:8002
- docker container exec -it {master container ID} /bin/bash
- apt-get update
- apt-get upgrade
- apt-get -y install curl
- curl http://worker1:80
- curl http://worker2:80
- docker compose down

Manager & Workers (Ubuntu) - 3/4

- FYI: DNS?
  - Domain Name Service



Reference: https://cdome.comodo.com/what-is-dns.php

Manager & Workers (Ubuntu) - 4/4

- FYI: curl://?
  - Client URL
  - is a computer software project providing a library (libcurl) and command-line tool (curl) for transferring data using various protocols
  - supports a range of common network protocols, currently including HTTP, HTTPS, SOAP requests, FTP, FTPS, SCP, SFTP, TFTP, LDAP, DAP, DICT, TELNET, FILE, IMAP, POP3, SMTP and RTSP

Reference: https://github.com/curl/curl

HOMEWORK

Manager & Workers (Alpine) - 1/4

- docker-compose.yml

```yaml
version: '3.7'
services:
  manager:
    image: alpine:latest # https://hub.docker.com/_/alpine
    tty: true # https://docs.docker.com/engine/reference/run/
  worker1:
    image: nginx # https://hub.docker.com/_/nginx
    ports: # https://docs.docker.com/compose/compose-file/#ports
      - "8001:80"
  worker2:
    image: nginx # https://hub.docker.com/_/nginx
    ports: # https://docs.docker.com/compose/compose-file/#ports
      - "8002:80"
```

HOMEWORK

Manager & Workers (Alpine) - 2/4

- docker compose up -d
- docker container ls
- curl (or web browser) http://localhost:8001
- curl (or web browser) http://localhost:8002
- docker container exec -it {master container ID} /bin/sh
- apk add curl
- curl http://worker1:80
- curl http://worker2:80
- docker compose down

HOMEWORK

Manager & Workers (Alpine) - 3/4

- docker compose up -d --scale manager=4
- docker container ls -a
- docker compose ps
- docker compose logs
- docker compose down

```
testdrive — -bash — 150×27
[MacMini-ServerFarm:testdrive drsungwon$ docker-compose up -d --scale manager=4
 Creating testdrive_worker1_1 ... done
 Creating testdrive_manager_1 ... done
 Creating testdrive_manager_2 ... done
 Creating testdrive_manager_3 ... done
 Creating testdrive_manager_4 ... done
 Creating testdrive_worker2_1 ... done
[MacMini-ServerFarm:testdrive drsungwon$ docker container ls -a
 CONTAINER ID        IMAGE               COMMAND                  CREATED             STATUS              PORTS                   NAMES
 f9de8e54f4e5        nginx               "nginx -g 'daemon of…"   13 seconds ago      Up 12 seconds       0.0.0.0:8002->80/tcp    testdrive_worker2_1
 4d1bda613207        alpine:latest       "/bin/sh"                14 seconds ago      Up 12 seconds                               testdrive_manager_1
 9f5f244863b1        alpine:latest       "/bin/sh"                14 seconds ago      Up 12 seconds                               testdrive_manager_2
 71aed0b0d252        alpine:latest       "/bin/sh"                14 seconds ago      Up 11 seconds                               testdrive_manager_3
 f3ad2ffbf9f2        alpine:latest       "/bin/sh"                14 seconds ago      Up 12 seconds                               testdrive_manager_4
 86be6b79f52d        nginx               "nginx -g 'daemon of…"   14 seconds ago      Up 11 seconds       0.0.0.0:8001->80/tcp    testdrive_worker1_1
[MacMini-ServerFarm:testdrive drsungwon$ docker-compose ps
        Name                      Command               State          Ports
 ------------------------------------------------------------------------------
 testdrive_manager_1   /bin/sh                         Up
 testdrive_manager_2   /bin/sh                         Up
 testdrive_manager_3   /bin/sh                         Up
 testdrive_manager_4   /bin/sh                         Up
 testdrive_worker1_1   nginx -g daemon off;            Up      0.0.0.0:8001->80/tcp
 testdrive_worker2_1   nginx -g daemon off;            Up      0.0.0.0:8002->80/tcp
[MacMini-ServerFarm:testdrive drsungwon$ docker-compose logs
 Attaching to testdrive_worker2_1, testdrive_manager_1, testdrive_manager_2, testdrive_manager_3, testdrive_manager_4, testdrive_worker1_1
 MacMini-ServerFarm:testdrive drsungwon$
```

HOMEWORK

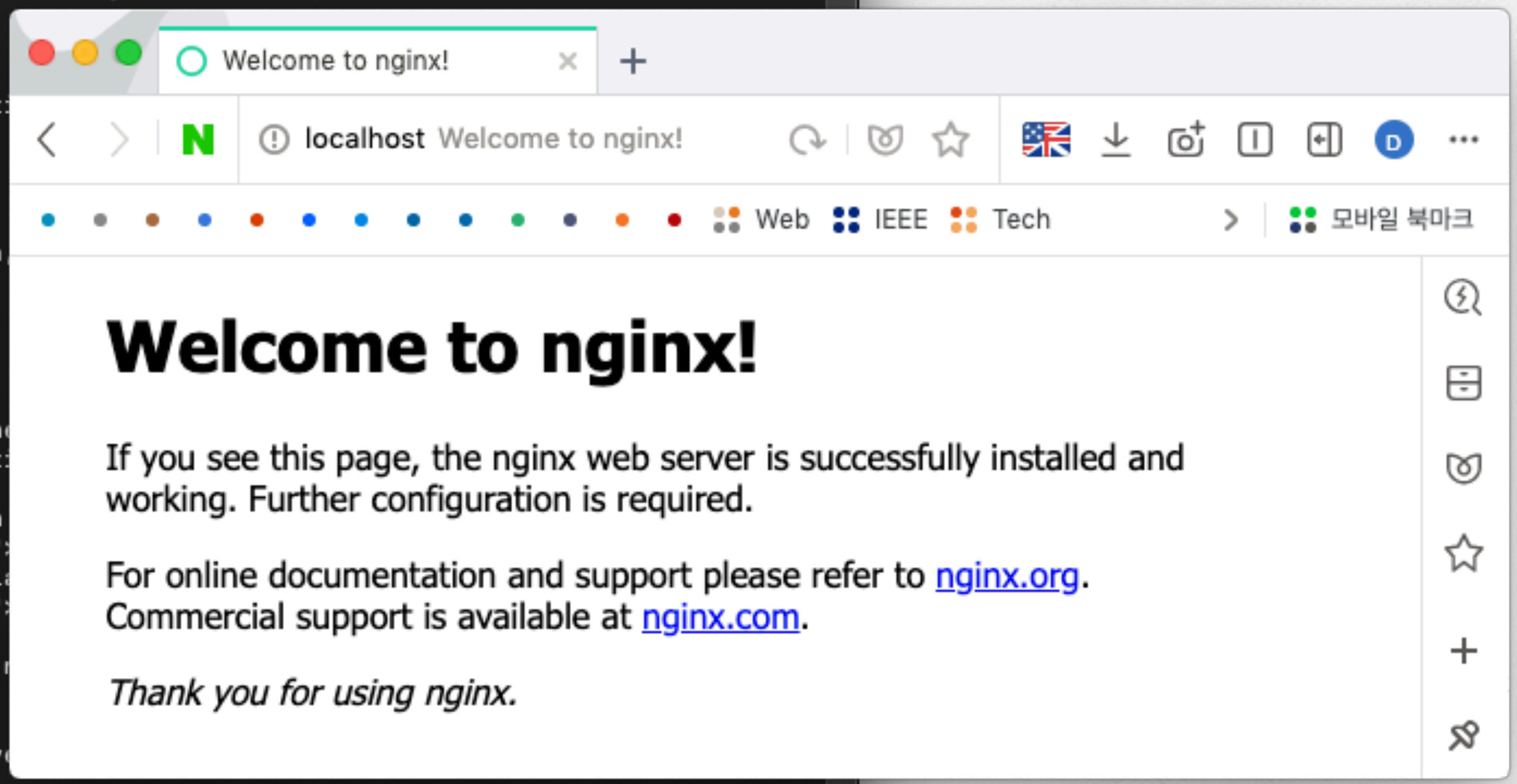
Manager & Workers (Alpine) - 4/4

- docker compose run -p 9998:80 -d worker1

```
testdrive — -bash — 82×31
[MacMini-ServerFarm:testdrive drsungwon$ docker-compose run -p 9998:80 -d worker1 ]
Creating network "testdrive_default" with the default driver
testdrive_worker1_run_8e675786057e
[MacMini-ServerFarm:testdrive drsungwon$ curl http://localhost:9998              ]
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</t
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, th
working. Further configurat

<p>For online documentation
<a href="http://nginx.org/"
Commercial support is avail
<a href="http://nginx.com/"

<p><em>Thank you for using
</body>
</html>
MacMini-ServerFarm:testdriv
```



Docker Compose Quickstart

- https://docs.docker.com/compose/gettingstarted/

References for Advanced Study

- Naver Cloud Center (30min)
  https://www.youtube.com/watch?v=qLFxB7Uk-o4

- Agile (19min)
  https://www.youtube.com/watch?v=NoMznX8S9pU

- CI/CD & DevOps (28min)
  https://www.youtube.com/watch?v=10TSLgh4gQM

- Docker Basic Re-visited (46min)
  https://www.youtube.com/watch?v=o4_KESBNFhI

Thank you

경희대학교
KYUNG HEE UNIVERSITY